Functional Dependency & Normalization

Functional Dependency

- A functional dependency occurs when the value of one attribute (or a set of attributes) uniquely determines the value of another attribute. This relationship is denoted as:

  $$\mathbf{X \rightarrow Y}$$

- Here, **X** is the determinant, and **Y** is the dependent attribute. This means that for each unique value of X, there is precisely one corresponding value of Y.

Example

StudentID	StudentName	StudentAge
101	Rahul	23
102	Ankit	22
103	Aditya	22
104	Sahil	24
105	Ankit	23

The above table has following FDs:
StudentID -> StudentName
StudentID -> StudentAge

These FDs do not hold :

StudentName -> StudentAge or
StudentAge -> StudentName **.**

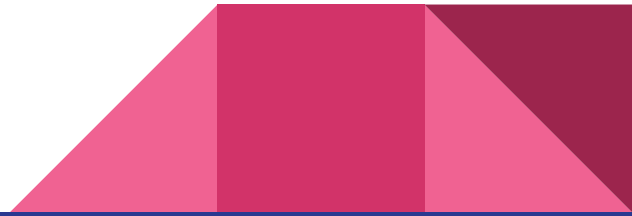
How to represent Functional Dependency in DBMS?

- Functional dependency is expressed in the form of equations. **Eg:** If we have an employee record with fields "EmployeeID", "FirstName" and "LastName" we can specify the function as follows:

EmployeeID -> FirstName, LastName

Benefits of Functional Dependency in DBMS

- The concept of normalization is based on functional dependencies. Using functional dependencies, we break a table in multiple tables that helps us in preventing duplicate data and hence Improves Data Quality, less errors and better database design.

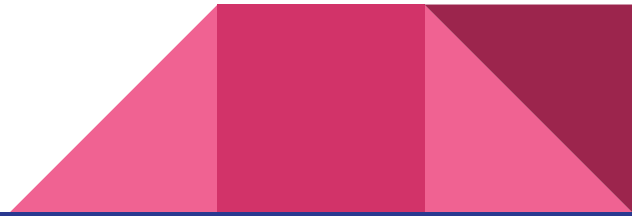


Types of Functional dependencies

- A functional_dependency occurs when one attribute uniquely determines another attribute within a relation.

1. Trivial Functional Dependency

- In Trivial Functional Dependency, a dependent is always a subset of the determinant. i.e. If X → Y and Y is the subset of X, then it is called trivial functional dependency.

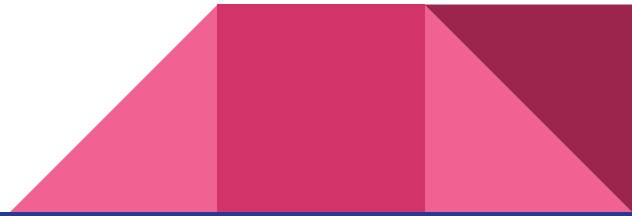


Eg 1: ABC -> AB
ABC -> A
ABC -> ABC

Symbolically: A→B is trivial functional dependency if B is a subset of A.

In simple terms : A **Trivial Functional Dependency** is a dependency that is **always true** because the right side is already part of the left side.

Here in FD, A->B, right side is B which is present in ABC->AB.
ABC → AB (Trivial)
Because AB is already inside ABC
ABC → A (Trivial)
Because A is part of ABC
ABC → ABC (Trivial)
Because both sides are the same



Eg 2:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, **{roll_no, name} → name** is a trivial functional dependency, since the right side (name) is already part of the left side (roll_no, name). If you already know **roll_no and name,** you obviously know name. So, no new information is added.

Similarly, **roll_no → roll_no** is also an example of trivial functional dependency. Both sides are the same. If you know roll_no, you already know roll_no.

2. **Non-trivial Functional Dependency**

- In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant. i.e. If **X → Y** and **Y is not a subset of X**, then it is called Non-trivial functional dependency.

- **Eg1 :** Id -> Name

Name -> DOB

Ans: Id → Name (Non-trivial)

Left side: **Id,** Right side: **Name**

Name is not part of Id

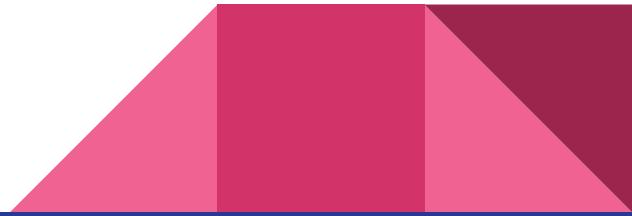
- If you know a student's **Id**, you can find their **Name**. This gives **new information**, so it is non-trivial.

Name → DOB (Non-trivial)

Left side: **Name,** Right side: **DOB**

DOB is not part of Name

- If you know a person's **Name**, you can determine their **Date of Birth**. Again, this gives **new information**.



Eg 2:

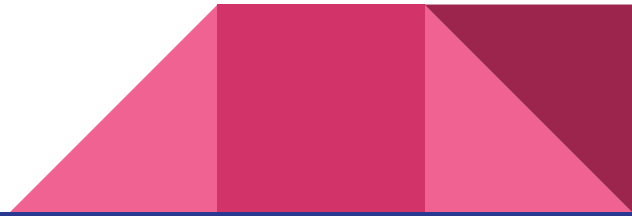
Roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

roll_no → name. This is **non-trivial** because **name is NOT part of roll_no**
If you know the **roll number**, you can find the **student's name**. This gives **new information.**

{roll_no, name} → age. This is also non-trivial because: age is NOT part of (roll_no, name). If you know roll_no and name, you can find the age. Again, this gives new information.

3. Multivalued Functional Dependency

- In Multivalued functional dependency, entities of the dependent set are not dependent on each other. i.e. If a → {b, c} and there exists no functional dependency between b and c, then it is called a multivalued functional dependency.



Example

bike_model	manuf_year	color
tu1001	2007	Black
tu1001	2007	Red
tu2012	2008	Black
tu2012	2008	Red
tu2222	2009	Black
tu2222	2009	Red

In this table:
X: bike_model
Y: color
Z: manuf_year

X->{Y,Z}

For a bike_model:

- It can have many colors
- It can have many manufacturing years

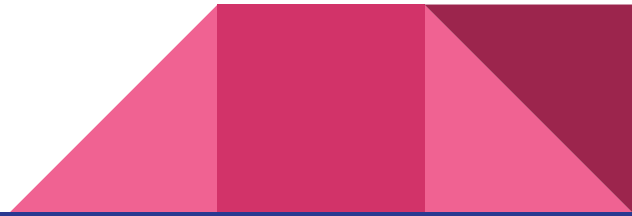
But:

- Color does NOT depend on manufacturing year
- Manufacturing year does NOT depend on color

The sets of color and manuf_year are linked only to bike_model.

4. Transitive Functional Dependency

- In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If a → b & b → c, then according to axiom of transitivity, a → c. This is a transitive functional dependency.



Example

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

enrol_no → dept
dept → building_no
According to the axiom of transitivity, enrol_no → building_no is a valid functional dependency.

5. Fully Functional Dependency

StudentID	CourseID	Marks
100	c100	85
101	c101	95
102	c102	75

In full functional dependency an attribute or a set of attributes uniquely determines another attribute or set of attributes.

(StudentID, CourseID) → Marks

A student can take many courses. Each course can have many students. So, to find Marks, we need StudentID, CourseID.

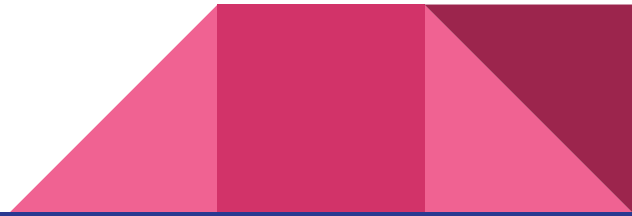
If we Remove CourseID → We don't know which course → cannot find Marks

If we Remove StudentID → We don't know which student → cannot find Marks.

Marks depends on both StudentID AND CourseID together.

Partial Functional Dependency

- In partial functional dependency a **non key attribute depends on a part of the composite key**, rather than the whole key. If a relation R has attributes X, Y, Z where X and Y are the composite key and Z is non key attribute. Then X->Z is a partial functional dependency in RDBMS.



Eg: (StudentID, CourseID) → StudentName
Here, **StudentName depends only on StudentID.** So we can write:
StudentID → StudentName

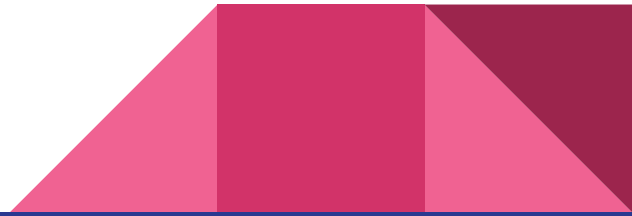
Why is it Partial?

- The key is **(StudentID, CourseID)** (composite key)
- But **StudentName** does NOT need CourseID
- It depends on **only part of the key (StudentID).** Not on the full key. That is we are using **extra attribute (CourseID)** which is not needed.

One student has **only one name**

- So, knowing **StudentID alone is enough**
- No need for CourseID

Full Dependency: Needs whole key.
Partial Dependency: Needs only part of key



Rule

If removing any attribute from the left side breaks the dependency →

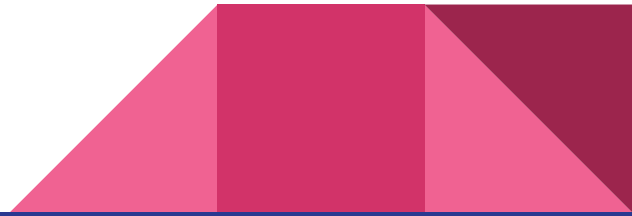
✔ Fully Functional Dependency

If dependency still holds after removing part of left side →

✘ Partial Dependency

Closure Method

- It helps to find all the candidate key in the table.
- Closure of an attribute x is the set of all attributes that are functional dependencies on X with respect to F. It is denoted by X+ which means what X can determine.



Example: Consider a relation R(A,B,C,D,E,F)
F: E->A, E->D, A->C, A->D, AE->F, AG->K.
Find the closure of E or E+
Solution:

$E^+ = E$
=EA {for E->A add A}
=EAD {for E->D add D}
=EADC {for A->C add C}
=EADC {for A->D D already added}
=EADCF {for AE->F add F}
=EADCF {for AG->K don't add k AG $\notin D^+$)

Eg: R (A B C D E), **FD =** { A → B, BC → D, E → C, D → A }

Solution: Here, we need to examine the attributes on the right-hand side of the functional dependencies, which are B, D, C, and A. Any attribute that does not appear on the right-hand side must appear on the left-hand side. Such attributes are essential and will be used in forming the candidate key.

Now E^+ = EC. Here E is not candidate key but E will be used to make candidate key.

Now E will be present with A, so closure of AE is:
$(AE)^+$ = A B E C D (A closure, A → B, E → C, BC → D)
∴ **AE is candidate key**

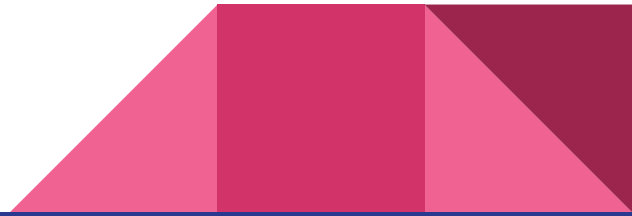
Q.1: Consider the relation scheme R = {E, F, G, H, I, J, K, L, M, N} and the set of functional dependencies {{E, F} -> {G}, {F} -> {I, J}, {E, H} -> {K, L}, K -> {M}, L -> {N} on R. What is the key for R? (GATE-CS-2014)

A. {E, F}

B. {E, F, H}

C. {E, F, H, K, L}

D. {E}



Solution:

Finding attribute closure of all given options, we get:
$\{E,F\}^+ = \{EFGIJ\}$
$\{E,F,H\}^+ = \{EFHGIJKLMN\}$
$\{E,F,H,K,L\}^+ = \{\{EFHGIJKLMN\}$
$\{E\}^+ = \{E\}$
$\{EFH\}^+$ and $\{EFHKL\}^+$ results in set of all attributes, but EFH is minimal. So it will be candidate key. So correct option is (B).

Q.4: Consider a relation scheme R = (A, B, C, D, E, H) on which the following functional dependencies hold: {A–>B, BC–> D, E–>C, D–>A}. What are the candidate keys of R? [GATE 2005]

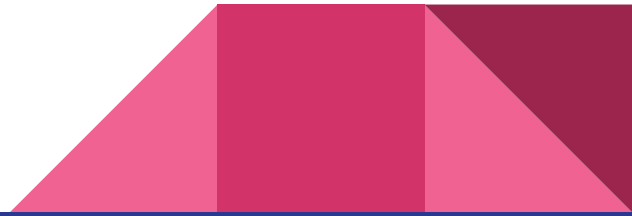
(a) AE, BE
(b) AE, BE, DE
(c) AEH, BEH, BCH
(d) AEH, BEH, DEH

Solution:

$(AE)^+$ = {ABECD} which is not set of all attributes. So AE is not a candidate key. Hence option A and B are wrong.
$(AEH)^+$= {ABCDEH}
$(BEH)^+$ = {BEHCDA}
$(BCH)^+$= {BCHDA} which is not set of all attributes. So BCH is not a candidate key. Hence option C is wrong.
So correct answer is D.

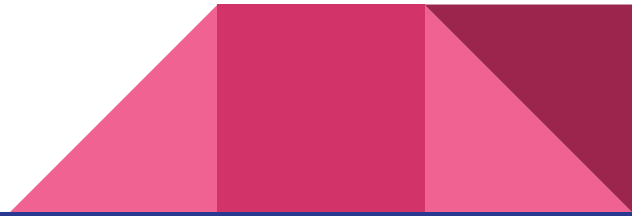
Prime attributes

- A **prime attribute** is an attribute that is part of a candidate key. Candidate keys are sets of attributes that uniquely identify tuples in a table. Therefore, prime attributes are those that can be used to uniquely identify any record in the table.

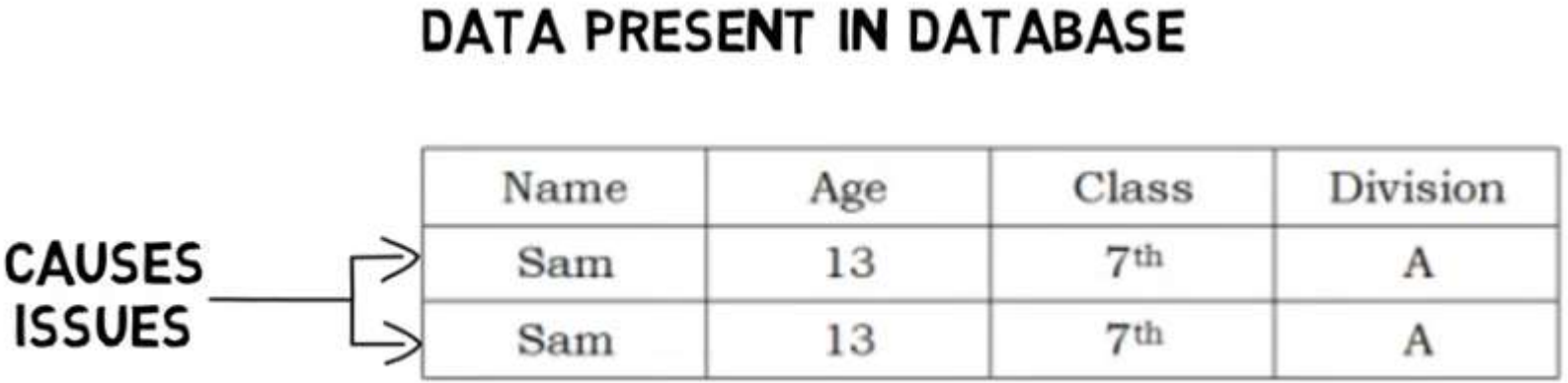


Non-Prime Attributes

- **Non-Prime Attributes**: Cannot be used to uniquely identify a tuple in the table because they can have the same values. They are often known as non-key attributes.



Normalization



Example: Students having same type of bag

DEFINITION:-

NORMALIZATION IS THE PROCESS OF REMOVING OR MINIMIZING REDUNDANCY FROM A TABLE.

Name	Age	Class	Division
Sam	13	7th	A
Sam	13	7th	A

IT CAN AFFECT:

- INSERTION
- UPDATION
- DELETION

Normalization try to remove redundancy or reduce the duplicate data

Eg: Redundancy

Table name: emp

	Employee_name	Department_name	Salary
	Kevin	Research	15,000
SAME	Michael	Production	18,000
	Samuel	Development	24,000
SAME	Michael	Production	18,000

UPDATE SALARY VALUE OF ROW 2 TO 20,000

UPDATE QUERY: update emp set Salary = 20000 where Employee_name = 'Michael' ;

Table name: emp

	Employee_name	Department_name	Salary
	Kevin	Research	15,000
SAME	Michael	Production	20,000
	Samuel	Development	24,000
SAME	Michael	Production	20,000

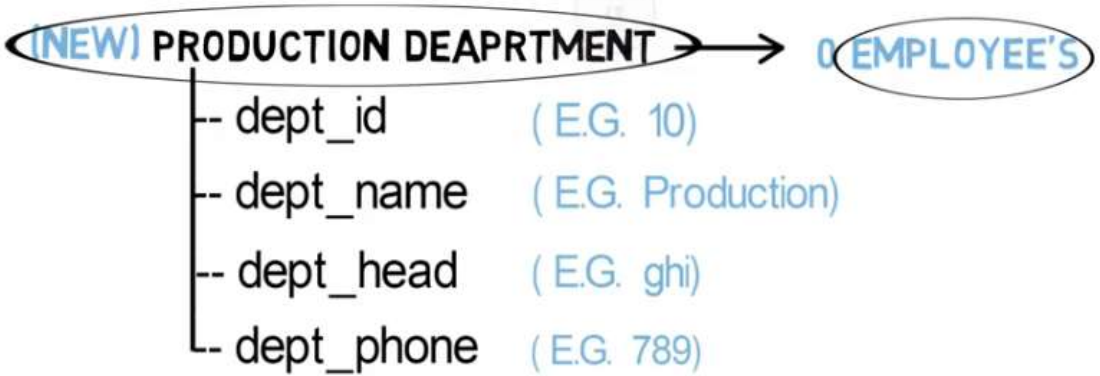
These kind of problems arise when there are redundancy in a table

Anomalies due to redundancy:

Insertion Anomaly

P.K.

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123
?			10	production	ghi	789



Delete Anomaly

Table Name: emp

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123
3	xyz	26	10	Production	tuv	987

Table Name: emp

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123

???

Delete Query:

```
delete from emp where id = 3;
```

Since there is only one employee in Production dept, when you apply the above query department information also gets deleted

Update Anomaly

Table Name: emp

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123
3	xyz	26	10	Production	tuv	987
4	slr	40	10	Production	tuv	987
5	smg	35	3	R & D	ksi	555
6	dmr	29	3	R & D	ksi	555

If R&D Dept Head is changed from ksi to msd

- In our example, we have a limited data, so we can easily identify the changes where and all are required.
- But in large database, it consists of many rows
- So, if any of the row is not updated it will result in inconsistency in database

Table Name: emp

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123
3	xyz	26	10	Production	tuv	987
4	slr	40	10	Production	tuv	987
5	smg	35	3	R & D	1 msd	555
6	dmr	29	3	R & D	2 ksi	555

Divide table into Emp and Dept

Table Name: emp

e_id	name	age	dept_id	dept_name	dept_head	dept_phone
1	abc	34	7	Design	pqr	123
2	mno	27	7	Design	pqr	123
3	xyz	26	10	Production	tuv	987
4	slr	40	10	Production	tuv	987
5	smg	35	3	R & D	ksi	555
6	dmr	29	3	R & D	ksi	555

HOW NORMALIZATION CAN SOLVE THESE ANOMALIES?

Table Name: emp

e_id	name	age	dept_id
1	abc	34	7
2	mno	27	7
3	xyz	26	10
4	slr	40	10
5	smg	35	3

F.K.

Table Name: dept

dept_id	dept_name	dept_head	dept_phone
7	Design	pqr	123
10	Production	tuv	987
3	R & D	ksi	555

Table Name: emp

F.K.

e_id	name	age	dept_id
1	abc	34	7
2	mno	27	7
3	xyz	26	10
4	slr	40	10
5	smg	35	3

Table Name: dept

dept_id	dept_name	dept_head	dept_phone
7	Design	pqr	123
10	Production	tuv	987
3	R & D	ksi	555
12	Marketing	trp	789

1) INSERTION ANOMALY

NEW DEAPRTMENT:- MARKETING

(NO EMPLOYEES)

Since the tables are divided, inserting new department data with no employees will not result in an insertion anomaly.

Table Name: emp

F.K.

e_id	name	age	dept_id
1	abc	34	7
2	mno	27	7
3	xyz	26	10
4	slr	40	10

Table Name: dept

dept_id	dept_name	dept_head	dept_phone
7	Design	pqr	123
10	Production	tuv	987
3	R & D	ksi	555
12	Marketing	trp	789

SMG got deleted from the emp table in database without deleting data in dept table

2) DELETION ANOMALY

SMG LEFT THE COMPANY

Table Name: emp

F.K.

e_id	name	age	dept_id
1	abc	34	7
2	mno	27	7
3	xyz	26	10
4	slr	40	10

Table Name: dept

dept_id	dept_name	dept_head	dept_phone
7	Design	mpr	123
10	Production	tuv	987
3	R & D	ksi	555
12	Marketing	trp	789

3) UPDATION ANOMALY

PQR ⟶ MPR

Candidate Key:

- A **Candidate Key** is a **minimal super key**.
- It **uniquely identifies each row**.
- **No attribute can be removed** from it without losing uniqueness.

Example Table: STUDENT

RollNo	Name	Email	Dept
101	Ravi	ravi@mail.com	CSE
102	Anu	anu@mail.com	ECE

Candidate Keys:

- {RollNo}
- {Email}

Why?

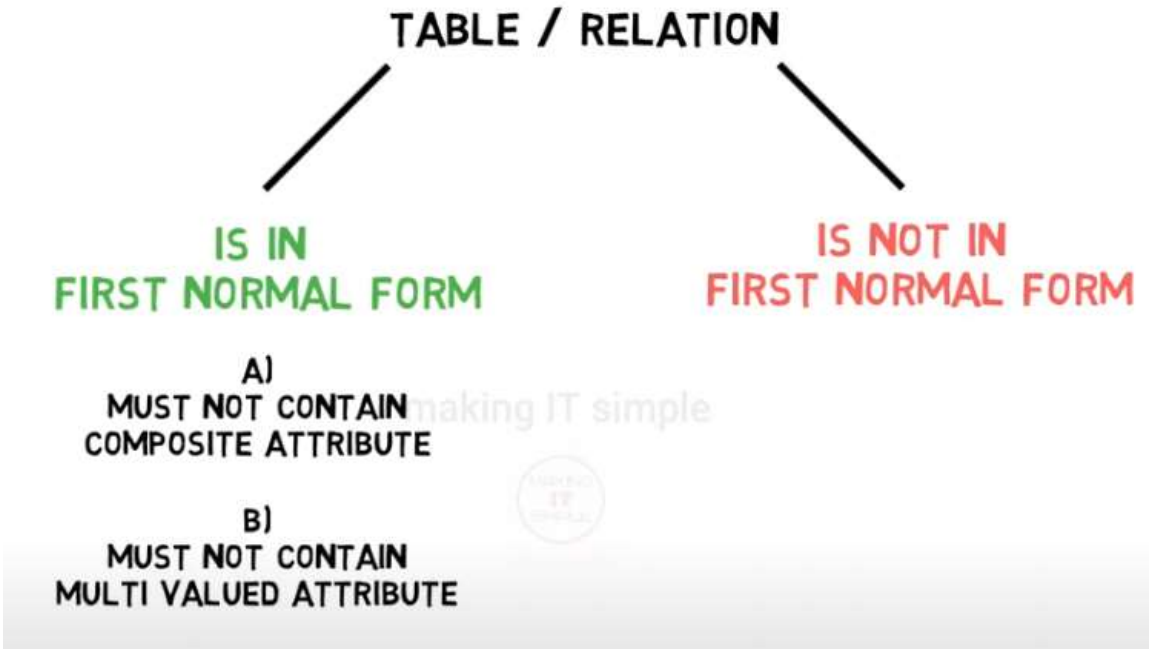
- Both uniquely identify each record.
- If you remove the attribute, uniqueness is lost.

But:

- {RollNo, Name} ❌ Not a candidate key

  because **RollNo alone is enough**, so **Name is unnecessary**.

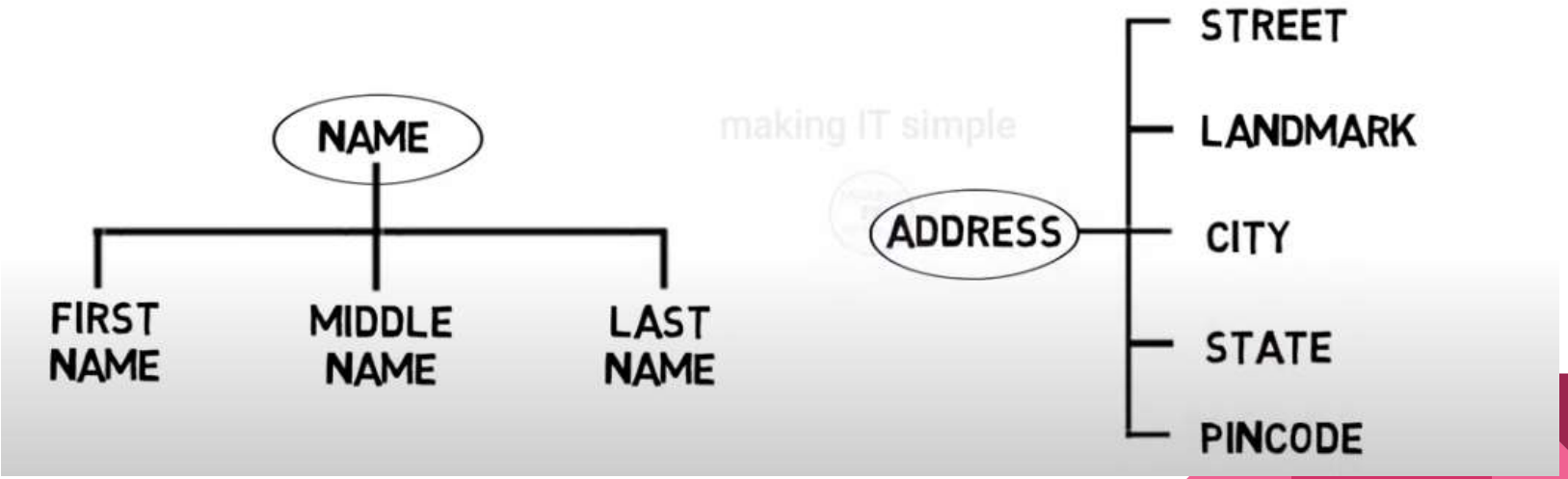
First Normal Form:



A)
MUST NOT CONTAIN
COMPOSITE ATTRIBUTE ?

B)
MUST NOT CONTAIN
MULTI VALUED ATTRIBUTE

Composite Attribute:
It is an attribute which can be further divided into parts



B)
MUST NOT CONTAIN
MULTI VALUED ATTRIBUTE ?

Multi valued Attribute:
It is an attribute which can have multiple values for single entity

Check whether the below table is in 1NF?

NO

ID	Name	Mobile Number
1	ABC DEF GHI	12345678, 876554321
2	JKL MNO PQR	11223344
3	RST UVW XYZ	21436587, 78563412

Yes, the below table is in 1NF

ID	First Name	Middle Name	Last Name	Mob Number	Alternate Mob No.
1	ABC	DEF	GHI	12345678	876554321
2	JKL	MNO	PQR	11223344	NULL
3	RST	UVW	XYZ	21436587	78563412

WHAT TO DO IF TABLE IS NOT IN FIRST NORMAL FORM?

COMPOSITE ATTRIBUTE

ID	Name	Mobile Number

ID	First Name	Middle Name	Last Name	Mob Number

Employee ID	Address
142	105, GM Street, Mumbai, 400012

Employee ID	House No.	Street Name	City	Pin-code
142	105	GM Street	Mumbai	400012

WHAT TO DO IF TABLE IS NOT IN FIRST NORMAL FORM?

MULTIVALUED ATTRIBUTE

ID	Name	Mobile Number
1	ABC	12345678, 876554321
2	JKL	11223344, 12442135
3	RST	21436587, 78563412
4	SMR	36925814, 74854875, 14251542
5	ASM	24867159

P.K.

ID	Name
1	ABC
2	JKL
3	RST
4	SMR
5	ASM

F.K.

ID	Mob Number	Employee ID
1	12345678	1
2	876554321	1
3	11223344	2
4	12442135	2
5	21436587	3
6	78563412	3
7	36925814	4
8	74854875	4
9	14251542	4
10	24867159	5

FIRST NORMAL FORM

CONDITION 1:
NO MULTIVALUED ATTRIBUTE

CONDITION 2:
NO COMPOSITE ATTRIBUTE

SECOND NORMAL FORM

CONDITION 1:
TABLE SHOULD BE IN
FIRST NORMAL FORM

CONDITION 2:
TABLE MUST NOT CONTAIN
PARTIAL DEPENDENCY.

CONDITION 1: TABLE MUST BE IN FIRST NORMAL FORM

1) NO MULTI-VALUED ATTRIBUTES

2) NO COMPOSITE ATTRIBUTES

ID	First Name	Last Name	Street	City	Pincode
1	Clark	Kent	Cardinal Street	Lorton	VA 2079
2	Tony	Stark	Valley Lane	Duarte	NY 11710
3	Bruce	Wayne	Gregory Street	Mokena	IL 60448

SECOND NORMAL FORM

TABLE SHOULD BE IN FIRST NORMAL FORM

TABLE MUST NOT CONTAIN PARTIAL DEPENDENCY.

CONDITION 1: IT MUST BE IN 1NF **CONDITION 2: THERE MUST NOT BE ANY PARTIAL DEPENDENCY**

Student_ID	Course_ID	Course_Fee
1	IOT	750
2	IOT	750
3	IOT	750
1	AI	880
2	AI	880
3	AI	880

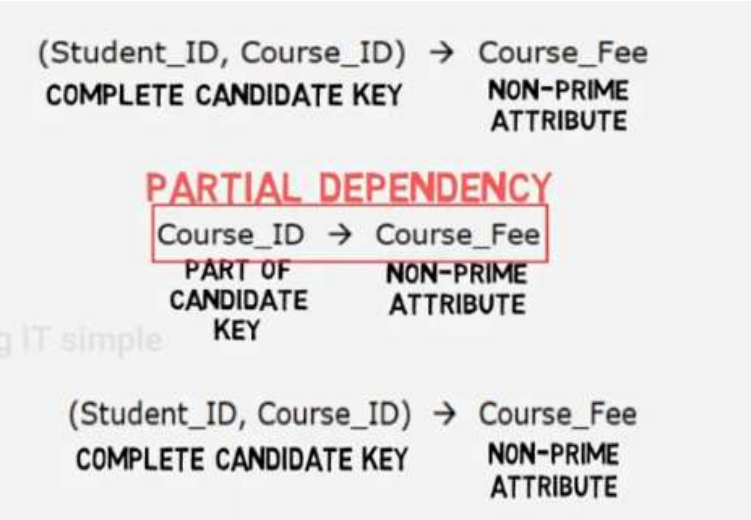
$(Student\_ID, Course\_ID)^+ = \{Student\_ID, Course\_ID, Course\_Fee\}$

(Student_ID, Course_ID)

IS A CANDIDATE KEY

Prime Attributes	Non-Prime Attributes
Student_ID	Course_Fee
Course_ID	

IF A NON-PRIME ATTRIBUTE IS DETERMINED BY A PART OF CANDIDATE KEY, THAT FUNCTIONAL DEPENDENCY IS PARTIAL DEPENDENCY



Functional dependency in the table:

Course_ID → Course_Fee

Course_fee depends on COURSE_ID not the whole key (Student_ID,Course_ID). So, this is called **Partial Dependency,** because non-prime attribute (course_fee) depends on part of Candidate key (Course_ID).

Because of **partial dependency**, the table **violates 2NF**.

Therefore, decompose the given table so that there is no partial dependency.

Split the given table, **Table 1: Student_Course** and **Table 2: Course**

Table 1: Student_Course

Student_ID	Course_ID	Course_Fee
1	IOT	750
2	IOT	750
3	IOT	750
1	AI	880
2	AI	880
3	AI	880

Student_ID	Course_ID
1	IOT
2	IOT
3	IOT
1	AI
2	AI
3	AI

Table 2: Course

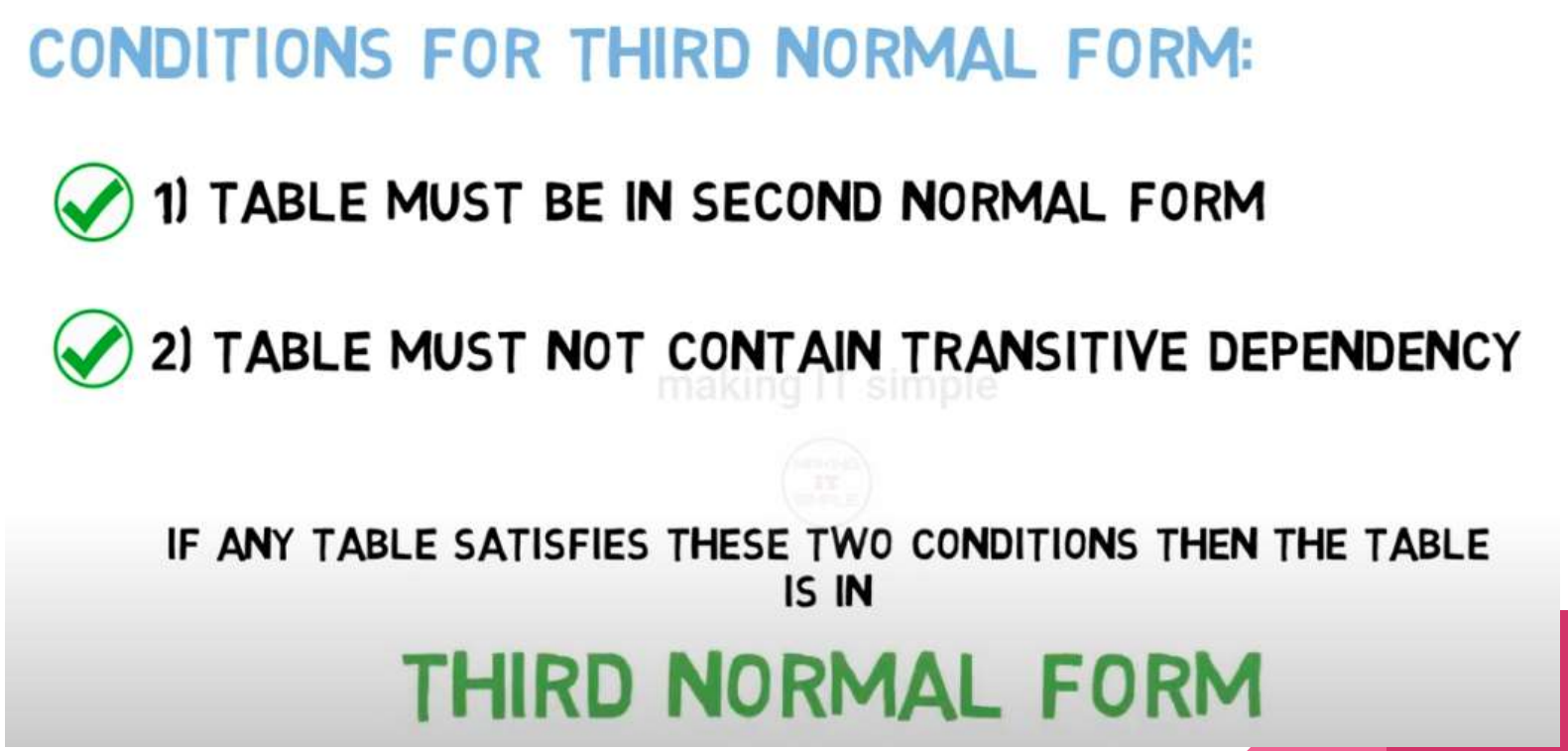
Course_ID	Course_Fee
IOT	750
AI	880

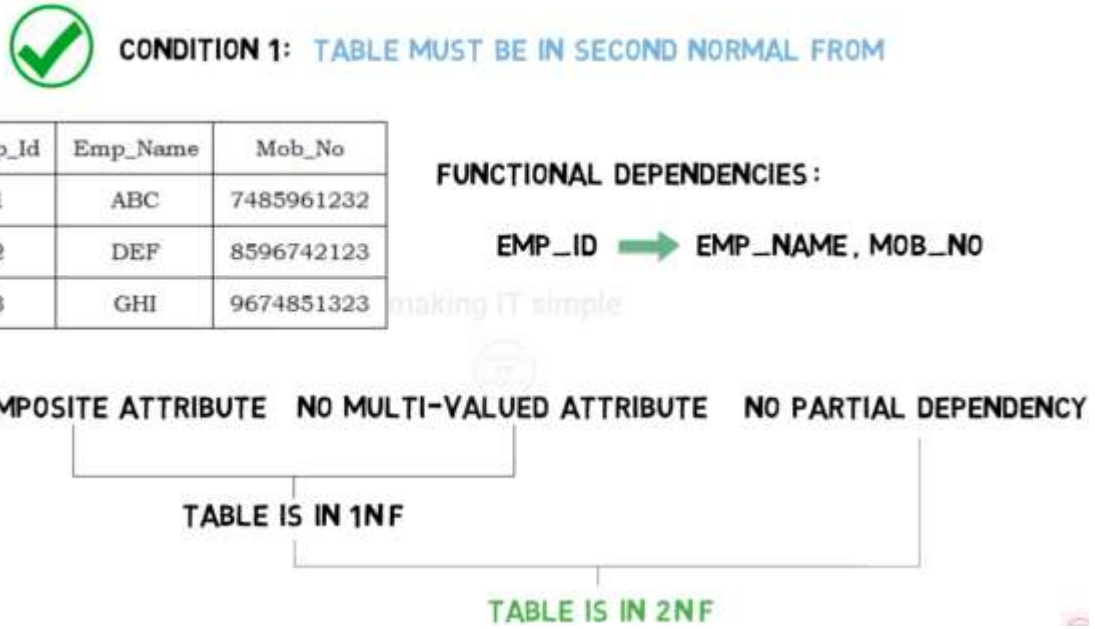
ELIMINATING REDUNDANCY IS THE AIM OF SECOND NORMAL FORM

Table 1 Primary Key: Student_ID, Course_ID. **Table 2 Primary Key:** Course_ID.
In Table 1 only key attribute is present. There is no non-prime attribute. Therefore, no partial dependency. So, this Table satisfies 2NF.

In Table 2 non key attribute depends on part of Composite key. Redundancy is removed. Therefore, there is no partial dependency. So, this Table satisfies 2NF.

THIRD NORMAL FORM (3NF):





The **primary key is a single attribute (Emp_Id)**, not composite. Therefore, **Partial dependency cannot exist**.

In 2NF- Partial dependency occurs when:

- The **primary key is composite (multiple attributes)**, and
- A **non-key attribute depends only on part of the primary key**.

What is Transitive dependency?

- A transitive dependency occurs when:
- Primary Key → Non-Key Attribute → Another Non-Key Attribute

From the given relation:

Emp_Id → Emp_Name

Emp_Id → Mob_No

Now check whether:

Emp_Name → Mob_No

or

Mob_No → Emp_Name

There is **no such dependency**.

Non-key attributes are:

- Emp_Name
- Mob_No

There is **no dependency between these two attributes**.

Emp_Name does not determine Mob_No

Mob_No does not determine Emp_Name

Possible transitive form would be:

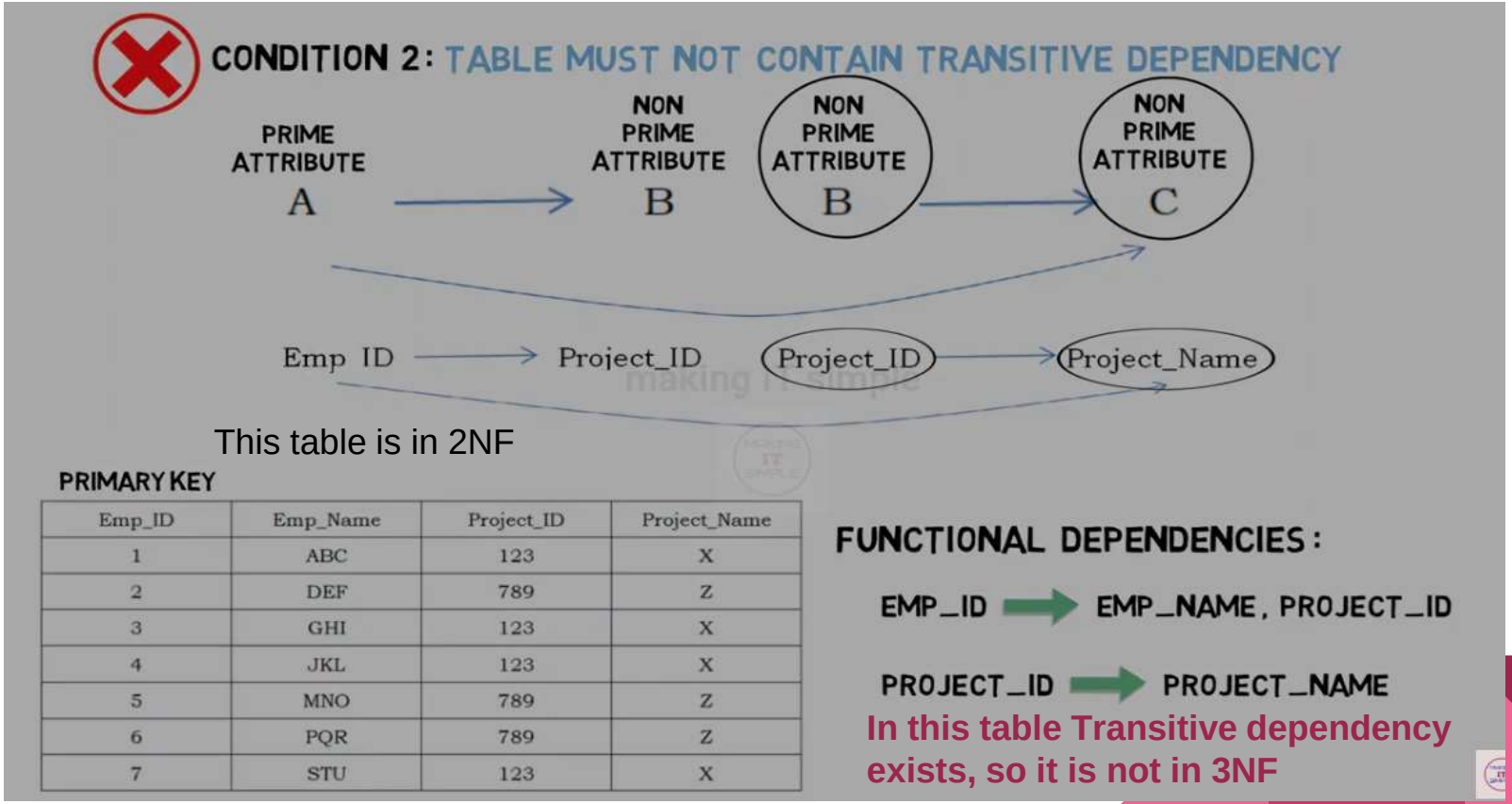
Emp_Id → Emp_Name → Mob_No

But this **does not exist**.

Thus **no indirect dependency** exists.

So, the table is in 3NF.

Example 2:


CONDITION 2: TABLE MUST NOT CONTAIN TRANSITIVE DEPENDENCY
PRIME ATTRIBUTE
A
NON PRIME ATTRIBUTE
B
NON PRIME ATTRIBUTE
B
NON PRIME ATTRIBUTE
C
Emp ID
Project_ID
Project_ID
Project_Name
This table is in 2NF
PRIMARY KEY
Emp_ID | Emp_Name | Project_ID | Project_Name
1 | ABC | 123 | X
2 | DEF | 789 | Z
3 | GHI | 123 | X
4 | JKL | 123 | X
5 | MNO | 789 | Z
6 | PQR | 789 | Z
7 | STU | 123 | X
FUNCTIONAL DEPENDENCIES:
EMP_ID → EMP_NAME, PROJECT_ID
PROJECT_ID → PROJECT_NAME
In this table Transitive dependency exists, so it is not in 3NF

Functional Dependencies: From the table we can identify:

Emp_ID → Emp_Name, Project_ID

- Each employee ID uniquely determines the Employee name and Project ID.

Project_ID → Project_Name

- Each project ID determines the project name.

Identify Prime and Non-Prime Attributes:

Prime Attribute: Attribute that is part of the **primary key**.

Prime Attribute:

- **Emp_ID**

Non-Prime Attributes

- **Emp_Name**
- **Project_ID**
- **Project_Name**

Emp_ID → Project_ID

Project_ID → Project_Name

Therefore,

Emp_ID → Project_ID → Project_Name

This creates an **indirect dependency**.

Divide Table into 2 Tables inorder to convert it to 3NF

Emp_ID	Emp_Name	Project_ID	Project_Name
1	ABC	123	X
2	DEF	789	Z
3	GHI	123	X
4	JKL	123	X
5	MNO	789	Z
6	PQR	789	Z
7	STU	123	X

THIS TABLE IS IN 2NF

Primary Key — Foreign Key

Emp_ID	Emp_Name	Project_ID
1	ABC	123
2	DEF	789
3	GHI	123
4	JKL	123
5	MNO	789
6	PQR	789
7	STU	123

Functional Dependencies:
Emp_ID → Emp_Name, Project_ID

WHY 3NF ???

IN HUGE DATABASE THIS WILL LEAD TO WASTE OF STORAGE

Primary Key

Project_ID	Project_Name
123	X
789	Z

Functional Dependencies:
Project_ID → Project_Name

What is Super key?

- A Super Key is any set of one or more attributes that can uniquely identify a tuple (row) in a table.
- It uniquely identifies each record.
- It may contain extra attributes that are not necessary.

Example Table: STUDENT

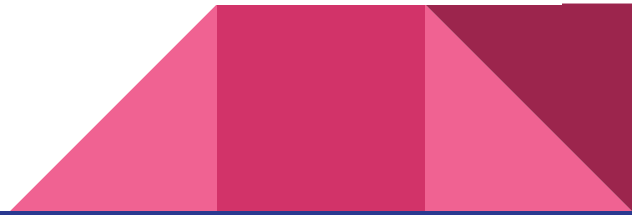
RollNo	Name	Email	Dept
101	Ravi	ravi@mail.com	CSE
102	Anu	anu@mail.com	ECE

Possible **Super Keys**:

- {RollNo}
- {Email}
- {RollNo, Name}
- {RollNo, Email}
- {RollNo, Name, Dept}

All these can uniquely identify a student.

But some contain **extra attributes**, so they are still super keys but **not minimal**.



(1) Find Superkey when F.D is given.

F.D: $A \to B$

$B \to C$

$C \to D$

Ans: Step 1: Find candidate key

$A^+ = A, B, C, D$

∴ A contains all attributes

So A is candidate key.

SATURDAY 08

WK 06 039-326

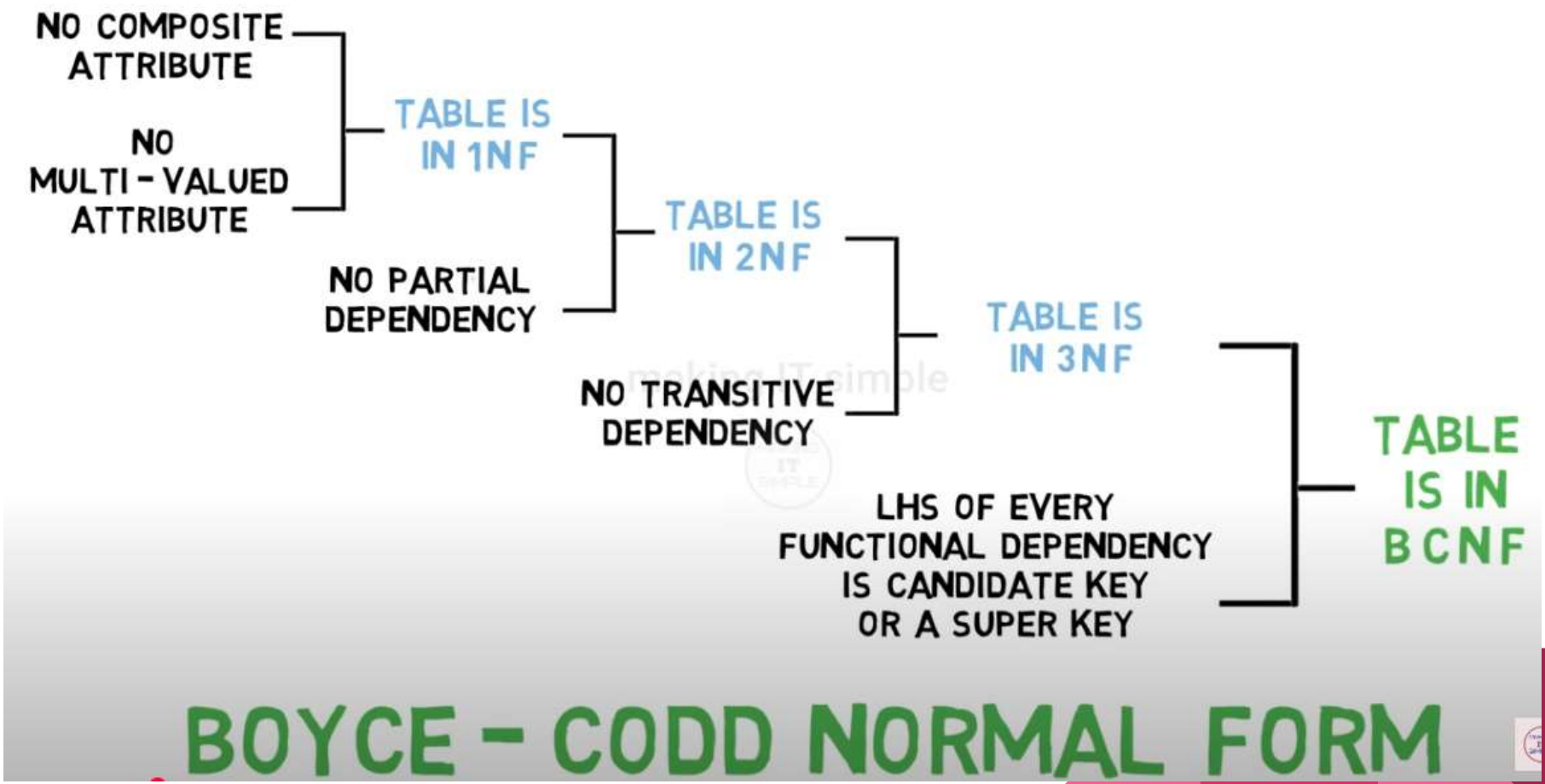
Step 2: Find superkey.

Since A is candidate key, any superset of A will also be a superkey.

∴ Superkey = $\{A\}, \{A, B\}, \{A, C\}, \{A, D\},$ SUNDAY 09

$\{A, B, C\}, \{A, B, D\}, \{A, C, D\},$

$\{A, B, C, D\}$


NO COMPOSITE ATTRIBUTE
NO MULTI-VALUED ATTRIBUTE
TABLE IS IN 1NF
NO PARTIAL DEPENDENCY
TABLE IS IN 2NF
NO TRANSITIVE DEPENDENCY
TABLE IS IN 3NF
LHS OF EVERY FUNCTIONAL DEPENDENCY IS CANDIDATE KEY OR A SUPER KEY
TABLE IS IN BCNF
BOYCE - CODD NORMAL FORM

CONDITIONS FOR BOYCE - CODD NORMAL FORM

1) TABLE MUST BE IN THIRD NORMAL FORM (3NF)

2) EVERY FUNCTIONAL DEPENDENCY IN THE TABLE SHOULD BE OF THIS FORM

STRICTLY A CANDIDATE OR SUPER KEY

Student	Subject	Teacher
ABC	Database	MNO
ABC	C	PQR
XYZ	Database	MNO
XYZ	C	STU

1NF

NO COMPOSITE ATTRIBUTE

NO MULTI-VALUED ATTRIBUTE

FUNCTIONAL DEPENDENCIES:

Student, Subject ⟶ Teacher

Teacher ⟶ Subject

composite key = (Student, Subject).

2NF ✓

NO PARTIAL DEPENDENCY

IS IN 1NF

PARTIAL DEPENDENCY

X ⟶ Y

PART OF CANDIDATE KEY — NON-PRIME ATTRIBUTE

Student, Subject ⟶ Teacher

COMPLETE CANDIDATE KEY — NON-PRIME ATTRIBUTE

PARTIAL DEPENDENCY

(LHS) X ⟶ Y

PART OF CANDIDATE KEY — NON-PRIME ATTRIBUTE

Teacher ⟶ Subject (RHS)

NON-PRIME ATTRIBUTE — PART OF CANDIDATE KEY

A table is in **3NF** if:

1. It is already in **2NF**.
2. **No non-prime attribute depends on another non-prime attribute.**

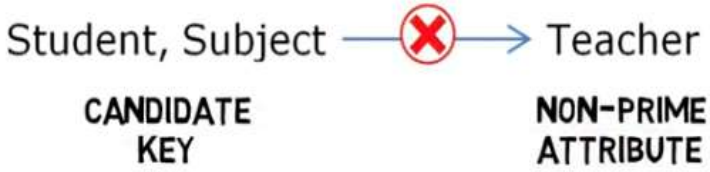
3NF
NO TRANSITIVE DEPENDENCY
IS IN 2NF

In simple words:

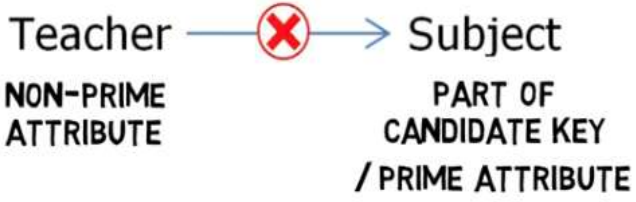
👉 **Non-key attributes should depend only on the primary key.**

✔ Transitive dependency only occurs when
Non-prime → Non-prime

TRANSITIVE DEPENDENCY



TRANSITIVE DEPENDENCY



CONDITION 1: Table should be in 3NF

CONDITION 2: EVERY FUNCTIONAL DEPENDENCY IN THE TABLE IS OF FORM X DETERMINES Y WHERE X OR LHS IS STRICTLY A CANDIDATE KEY OR A SUPER KEY.

Student	Subject	Teacher
ABC	Database	MNO
ABC	C	PQR
XYZ	Database	MNO
XYZ	C	STU

Candidate Key: {Student, Subject}

Functional Dependencies:

Student, Subject → Teacher

Teacher → Subject

(LHS) X → Y

CANDIDATE OR A SUPER KEY

Student, Subject → Teacher

CANDIDATE KEY

NON-PRIME ATTRIBUTE

CONDITION 2: EVERY FUNCTIONAL DEPENDENCY IN THE TABLE IS OF FORM X DETERMINES Y WHERE X OR LHS IS STRICTLY A CANDIDATE KEY OR A SUPER KEY.

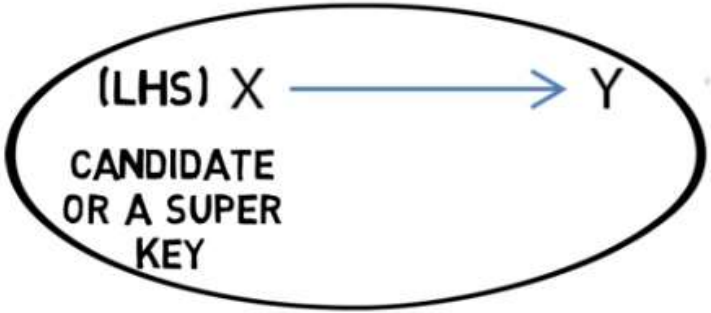
Student	Subject	Teacher
ABC	Database	MNO
ABC		PQR
XYZ	Database	MNO
XYZ	C	STU

Functional Dependencies:

Student, Subject → Teacher

Teacher → Subject

Candidate Key: {Student, Subject}



Divide Table into 2 Tables inorder to convert it to BCNF

Student	Subject	Teacher
ABC	Database	MNO
ABC	C	PQR
XYZ	Database	MNO
XYZ	C	STU

Candidate Key: {Student, Subject}

Functional Dependencies:

Student, Subject → Teacher

Teacher → Subject ✗

R {STUDENT, SUBJECT, TEACHER}

R1
{STUDENT, TEACHER}
FK

R2
{TEACHER, SUBJECT}
PK

BOYCE - CODD NORMAL FORM